

Week 8: Genetic Algorithm

Lab Duration: 2 Hours 10 Minutes

Lab Objectives

By completing this lab, students will be able to:

1. Understand the fundamental GA concepts: **chromosome representation, population, fitness, selection, crossover, mutation.**
2. Implement GA for **binary, real-valued, and combinatorial optimization problems.**
3. Analyze the effect of **GA parameters** (population size, mutation rate, selection type) on convergence.
4. Apply GA to **realistic problem scenarios.**

Submission Guidelines:

1. Each task must properly show the output. **DO NOT** Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the obtained marks.**

Task 1: Binary String Maximization (Basic GA Concepts)

Problem: Maximize the number of 1's in a binary string of length 10.

Instructions:

1. **Population:** Generate 6 random binary strings, each of length 10.
2. **Fitness Function:** Count the number of 1's in a string.
3. **Selection:** Use **roulette wheel selection** to pick parents.
4. **Crossover:** Single-point crossover between two parents to produce two children.
5. **Mutation:** Flip a bit in a chromosome with 10% probability.
6. **Generations:** Run for **5 generations.**

Output:

- Display the population and fitness **each generation.**
- Highlight the **best chromosome per generation.**

Hints:

- Chromosome can be a Python list of 0's and 1's.
- Use `random.choices()` for roulette wheel selection.
- Single-point crossover: pick a random index and swap tails.

Task 2: Function Maximization (Continuous GA)

Problem: Maximize the function:

$$f(x) = x \cdot \sin(10\pi x) + 1.0, x \in [0,1]$$

Instructions:

1. **Chromosome:** Single float number representing x .
2. **Population:** 5-6 chromosomes initialized randomly in $[0,1]$.
3. **Fitness:** Evaluate $f(x)$ for each chromosome.
4. **Selection:** Tournament selection with **size 2**.
5. **Crossover:** Arithmetic crossover:

$$\text{child} = \alpha \cdot \text{parent1} + (1 - \alpha) \cdot \text{parent2}$$

- Use random α in $[0,1]$.
6. **Mutation:** Add Gaussian noise $N(0, 0.05)$ to a chromosome.
 7. **Generations:** Run for **10 generations**.

Output:

- Best x and $f(x)$ per generation.
- Plot best fitness vs. generations.

Hints:

- Use `random.gauss(0, 0.05)` for mutation.
- Tournament selection: pick two random chromosomes, select the one with higher fitness.

Task 3: 0/1 Knapsack Problem using GA

Problem: Optimize selection of items within weight limit.

Item	Weight	Value
1	2	3
2	3	4
3	4	5
4	5	8

- Max weight = 8

Instructions:

1. **Chromosome:** Binary string of length 4 (1 = item selected).
2. **Fitness:** Sum of values if total weight ≤ 8 , else 0.
3. **Selection:** Roulette wheel.
4. **Crossover:** Single-point crossover.
5. **Mutation:** Flip one random bit.
6. **Generations:** 10 generations.

Output:

- Best chromosome, total weight, total value per generation.

Hints:

- Calculate total weight first, apply **penalty if exceeded**.

*It helps you learn GA with **constraints**.*

Task 4: Traveling Salesman Problem (TSP) Simplified using GA

Problem: Find a near-optimal route for 5 cities with distance matrix:

	A	B	C	D	E
A	0	2	9	10	7
B	2	0	6	4	3
C	9	6	0	8	5
D	10	4	8	0	6
E	7	3	5	6	0

Instructions:

1. **Chromosome:** A permutation of [A,B,C,D,E].
2. **Fitness:** Inverse of total route distance.
3. **Selection:** Tournament selection (size 2).

4. **Crossover:** Order crossover (OX).
5. **Mutation:** Swap two cities in the route.
6. **Generations:** Run for 10 generations.

Output:

- Best route and total distance per generation.

*You will see **GA applied to permutations**, not just binary/continuous values.*

Task 5: Evolving Logical Expressions (Symbolic GA)

Problem: Maximize the boolean expression:

$$F(a, b, c) = (a \text{ AND } b) \text{ OR } (b \text{ AND } c)$$

Instructions:

1. **Chromosome:** List of 3 bits [a,b,c].
2. **Fitness:** Number of truth table rows where chromosome gives correct F(a,b,c).
3. **Population:** 6 chromosomes.
4. **Selection:** Roulette wheel.
5. **Crossover:** Single-point.
6. **Mutation:** Flip a random bit.
7. **Generations:** 5–6 generations.

Output:

- Show fitness of each chromosome per generation.
- Best solution chromosome per generation.

Learning:

- *You will learn how GA work for **logical/symbolic problems**.*

Task 6: GA for Simple Equation Solving

Problem: Find integers x and y that satisfy:

$$x^2 + y^2 = 50$$

Instructions:

1. **Chromosome:** $[x,y]$, integers in $[0,10]$.
2. **Fitness:** $1 / (1 + \text{abs}(x^2 + y^2 - 50))$.
3. **Population:** 6 chromosomes.
4. **Selection:** Tournament selection (size 2).
5. **Crossover:** Average of parents (rounded to nearest integer).
6. **Mutation:** Add or subtract 1 randomly from x or y .
7. **Generations:** 10 generations.

Output:

- Best solution and fitness per generation.

*You will see how GA work for **numerical equation solving**.*

Tips for Lab Tasks

1. **Start small:** Use a small population and fewer generations to understand GA evolution.
2. **Visualize:** Print chromosomes and fitness at each generation to “see” GA working.
3. **Experiment:** Change mutation rate, population size, selection type, observe impact.
4. **Abstract the code:** Write generic functions for selection, crossover, and mutation; reuse them across tasks.
5. **Think conceptually:** Focus on GA mechanics (fitness evaluation, exploration vs exploitation).